



Ingesting by Batch or by Stream

AWS Academy Data Engineering

Introduction

Ingesting by Batch or by Stream

Module objectives

This module prepares you to do the following:

- List key tasks that the data engineer needs to perform when building an ingestion layer.
- Describe how purpose-built AWS services support ingestion tasks.
- Illustrate how the features of AWS Glue work together to support automating batch ingestion.
- Describe AWS streaming services and features that simplify streaming ingestion.
- Identify configuration options in AWS Glue and Amazon Kinesis Data Streams that help you scale your ingestion processing to meet your needs.
- Describe distinct characteristics of ingesting Internet of Things (IoT) data by using AWS IoT Core.

Module overview

Presentation sections

- Comparing batch and stream ingestion
- Batch ingestion processing
- Purpose-built ingestion tools
- AWS Glue for batch ingestion processing
- Scaling considerations for batch processing
- Services for stream processing
- Scaling considerations for stream processing
- Ingesting IoT data by stream

Lab

- Performing ETL on a Dataset by Using AWS Glue

Knowledge checks

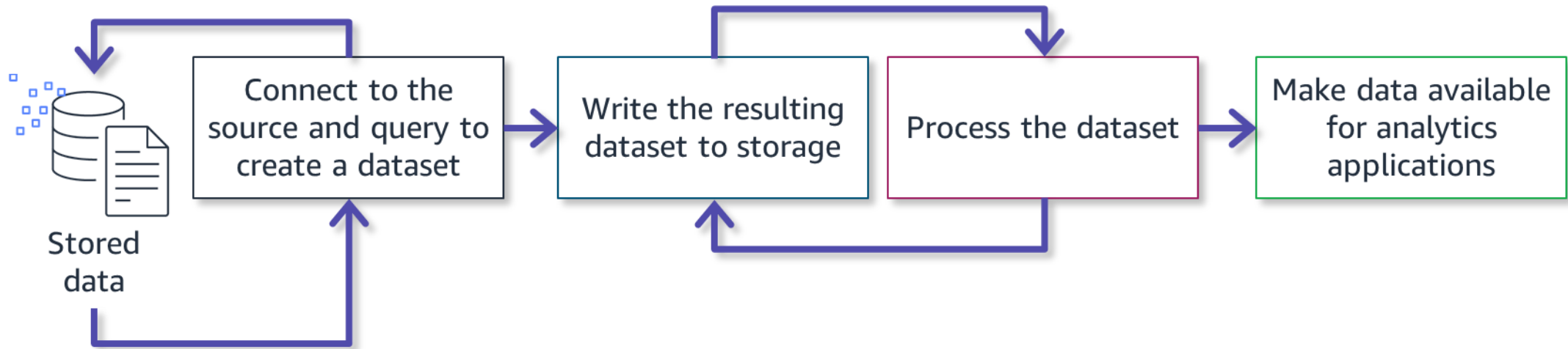
- Online knowledge check
- Sample exam question

Comparing batch and stream ingestion

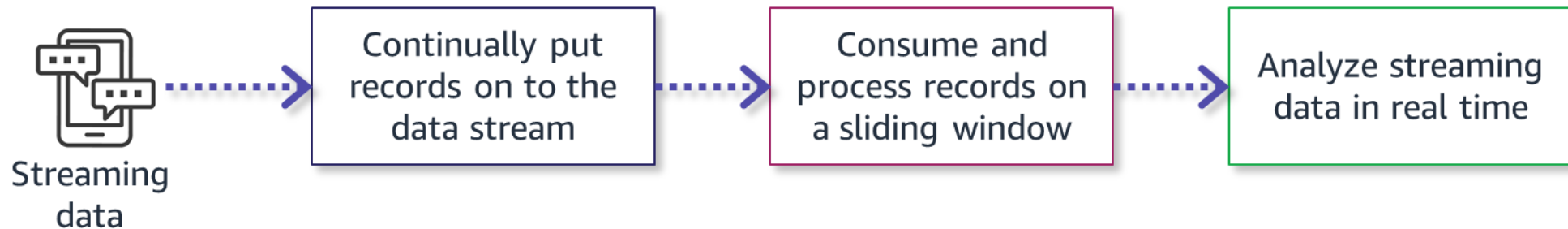
Ingesting by Batch or by Stream

Batch and streaming ingestion data flow

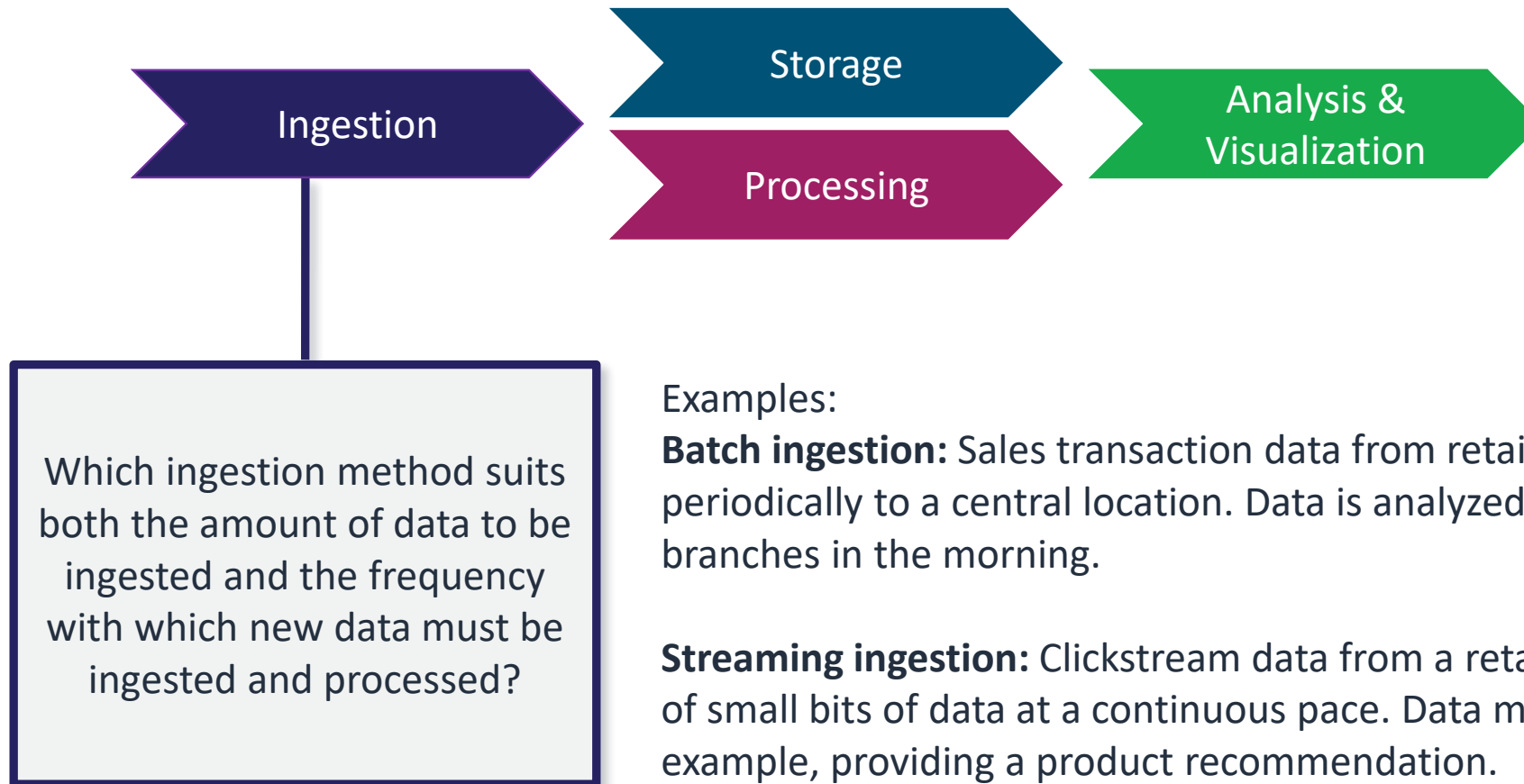
Batch: Ingest and process a batch of records as a dataset. Run on demand, on a schedule, or based on an event.



Streaming: Ingest records continually and process sets of records as they arrive on the stream.



Data volume and velocity are primary drivers



Examples:

Batch ingestion: Sales transaction data from retailers across the world is sent periodically to a central location. Data is analyzed overnight, and reports are sent to branches in the morning.

Streaming ingestion: Clickstream data from a retailer's website sends a large volume of small bits of data at a continuous pace. Data must be analyzed immediately. For example, providing a product recommendation.

Key takeaways: Comparing batch and stream ingestion



- Batch jobs query the source, transform the data, and load it into the pipeline.
- Traditional ETL uses batch processing.
- With stream processing, producers put records on a stream where consumers get and process them.
- Streams are designed to handle high-velocity data and real-time processing.

Batch ingestion processing

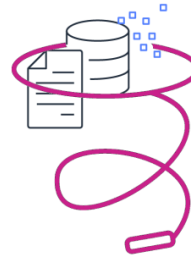
Ingesting by Batch or by Stream

Tasks to build a batch processing pipeline

Extract



Transform/Load



Load/Transform



Run
scripts
and jobs

Connect to sources
and select data

Identify the source
and target schemas

Transfer and store
data securely

Transform the
dataset

Load the dataset to
durable storage



Orchestrate workflows

Key characteristics for batch processing design choices

Ease of use	Data volume and variety	Orchestration and monitoring	Scaling and cost management
• Make it flexible. • Offer low-code or no-code options. • Offer serverless options.	• Handle large volumes of data. • Support disparate source and target systems. • Support different data formats seamlessly.	• Support workflow creation. • Provide dependency management on the workflow. • Support bookmarking. • Alert on job failure. • Enable logging.	• Enable automatic scaling. • Offer pay-as-you-go options.

Key takeaways: Batch ingestion processing

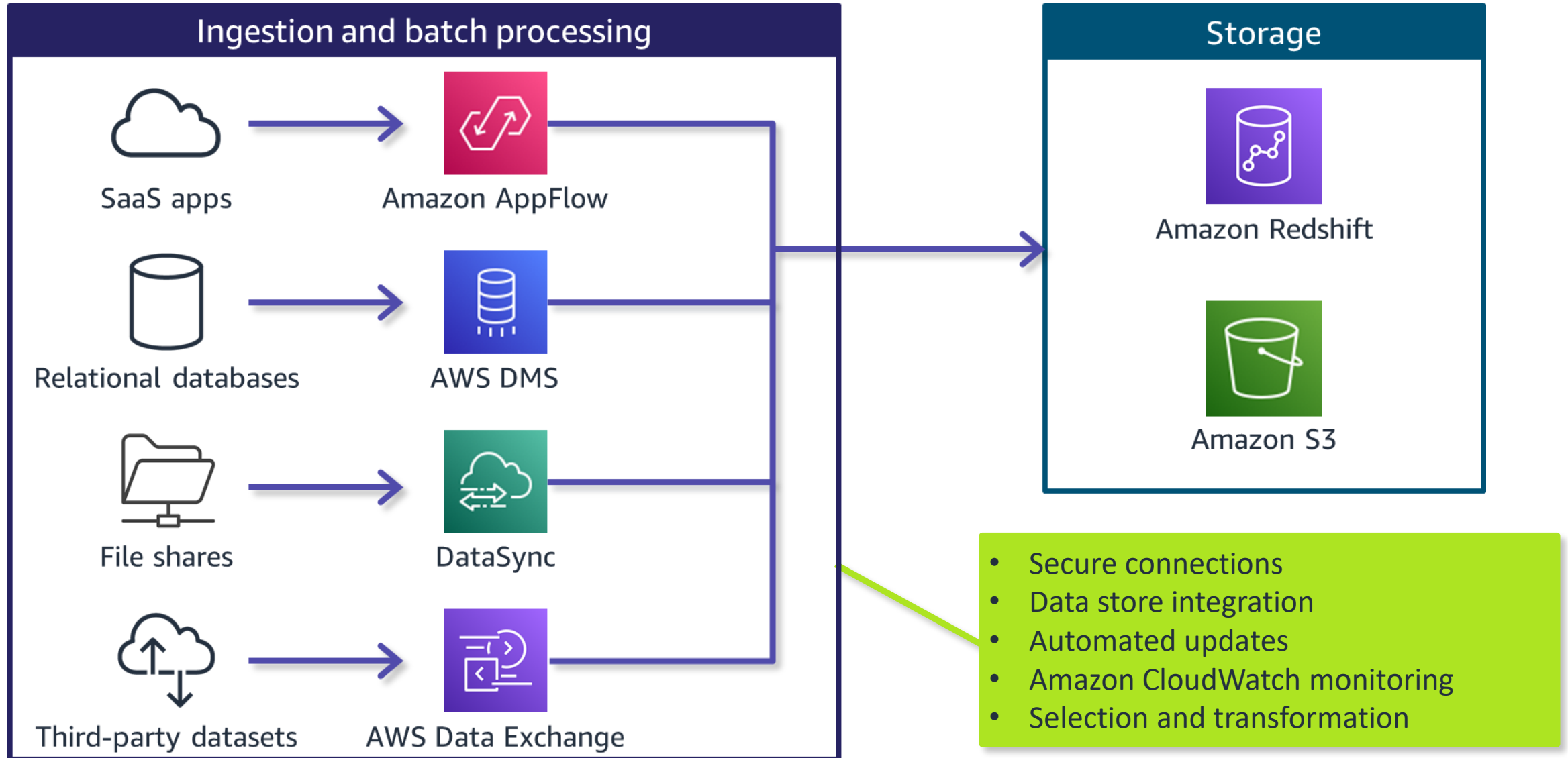


- Batch ingestion involves writing scripts and jobs to perform the ETL or ELT process.
- Workflow orchestration helps you to handle interdependencies between jobs and manage failures within a set of jobs.
- Key characteristics for pipeline design include ease of use, data volume and variety, orchestration and monitoring, and scaling and cost management.

Purpose-built ingestion tools

Ingesting by Batch or by Stream

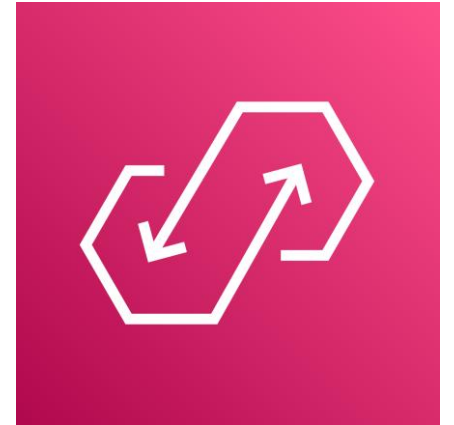
Match AWS purpose-built tools to data sources



Amazon AppFlow

Use Amazon AppFlow to ingest data from a **software as a service (SaaS)** application.

- Create a connector with filters.
- Map fields and perform transformations.
- Perform validation.
- Securely transfer to Amazon S3 or Amazon Redshift.



Example: Ingest customer support ticket data from Zendesk.

AWS Database Migration Service (AWS DMS)

Use AWS DMS to ingest data from your **relational databases**.

- Connect to source data and format it for a target.
- Use source filters and table mappings.
- Perform data validation.
- Write to many AWS data stores.
- Create a continuous replication task.



Example: Ingest line of business transactions from an Oracle database.

AWS DataSync

Use DataSync to ingest data from **file systems**.

- Apply filters to transfer a subset of files.
- Use a variety of file systems as sources and target, including Amazon S3 as a target.
- Securely transfer data between self-managed storage systems and AWS storage services.



Example: Ingest on-premises genome sequencing data to Amazon S3.

AWS Data Exchange

Use AWS Data Exchange to **integrate third-party datasets** into your pipeline.

- Find and subscribe to sources.
- Preview before subscribing.
- Copy subscribed datasets to Amazon S3.
- Receive notifications of updates.



Example: Ingest de-identified clinical data from a third party.

Key takeaways:

Purpose-built ingestion tools

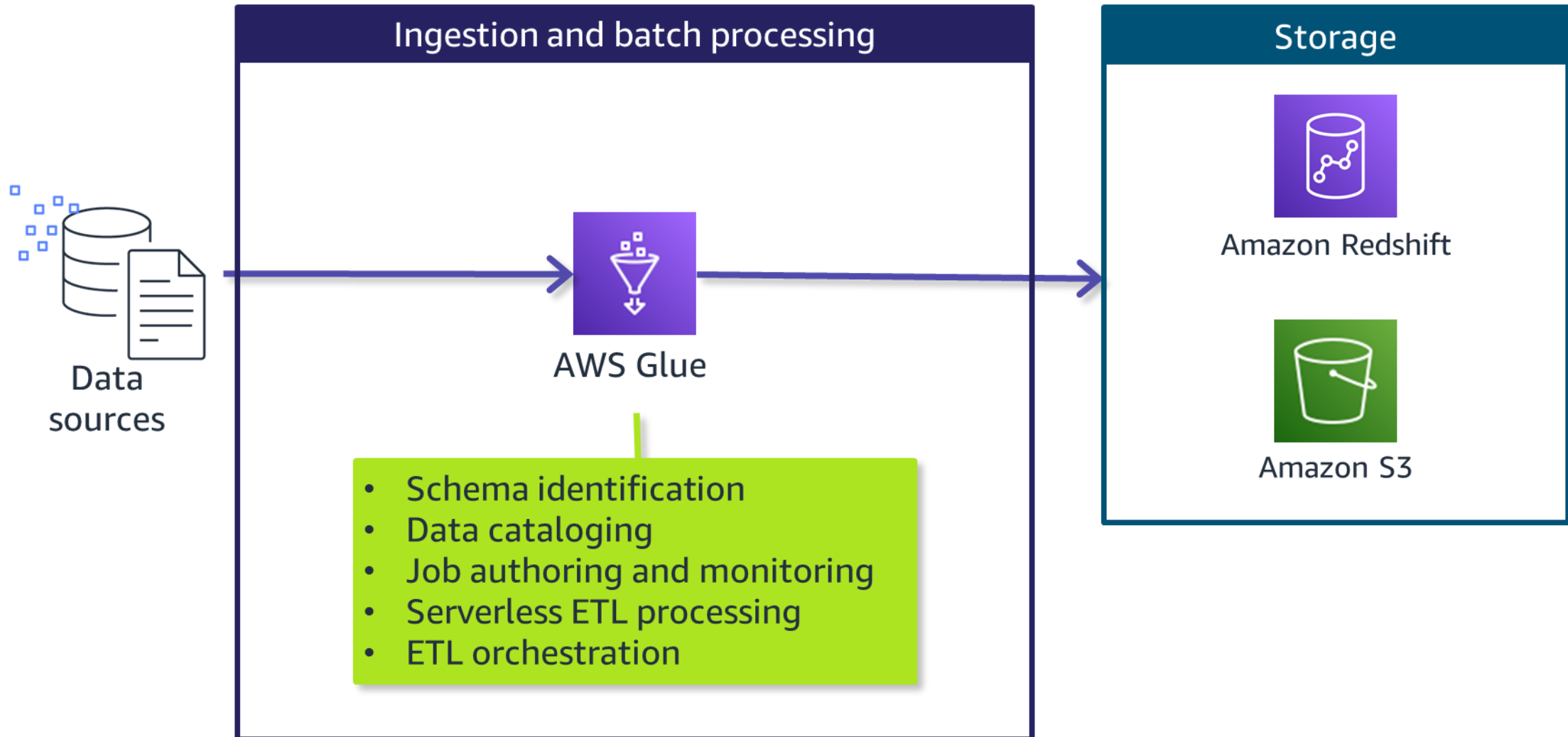


- Choose purpose-built tools that match the type of data to be ingested and simplify the tasks that are involved in ingestion.
- Amazon AppFlow, AWS DMS, and DataSync each simplify the ingestion of specific data types.
- AWS Data Exchange provides a simplified way to find and subscribe to third-party datasets.

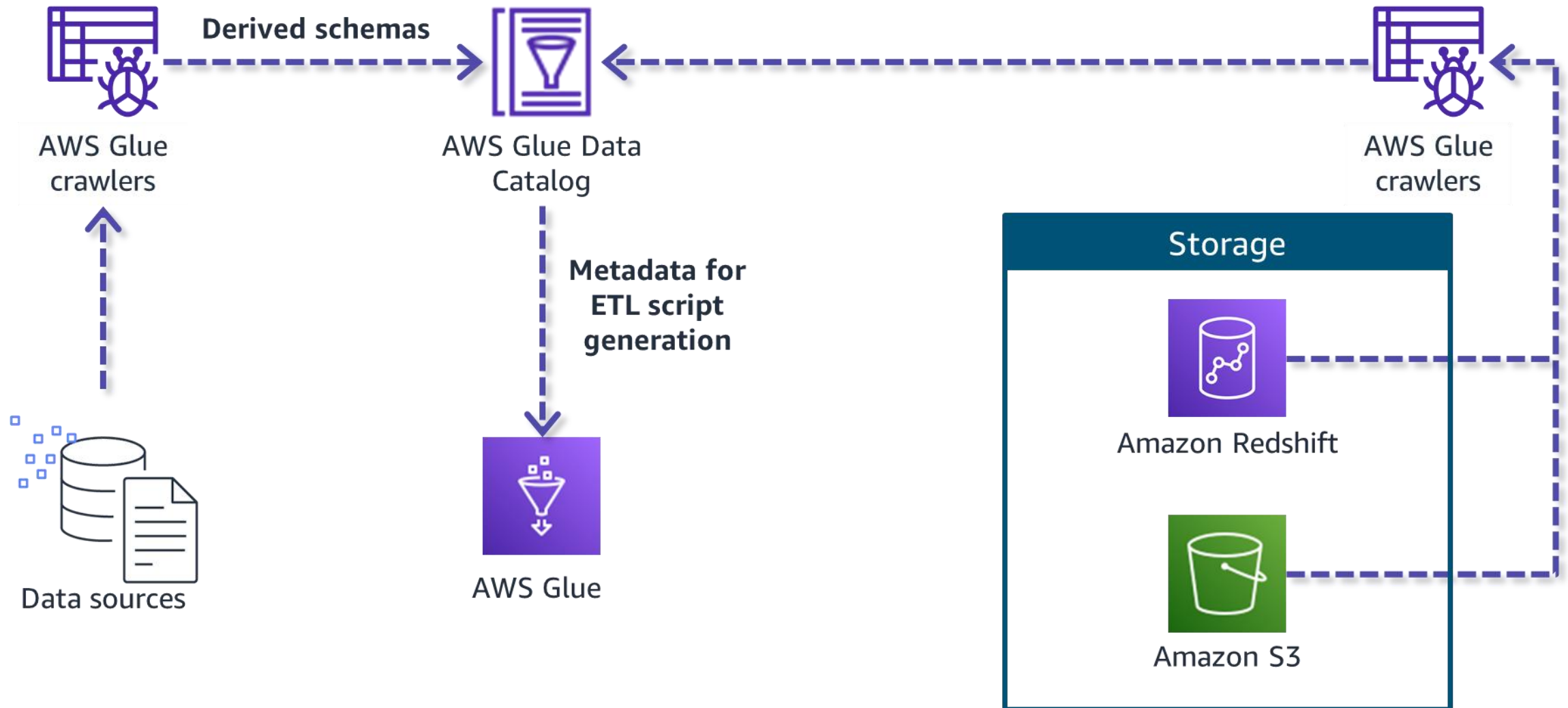
AWS Glue for batch ingestion processing

Ingesting by Batch or by Stream

AWS Glue simplifies batch ingestion tasks

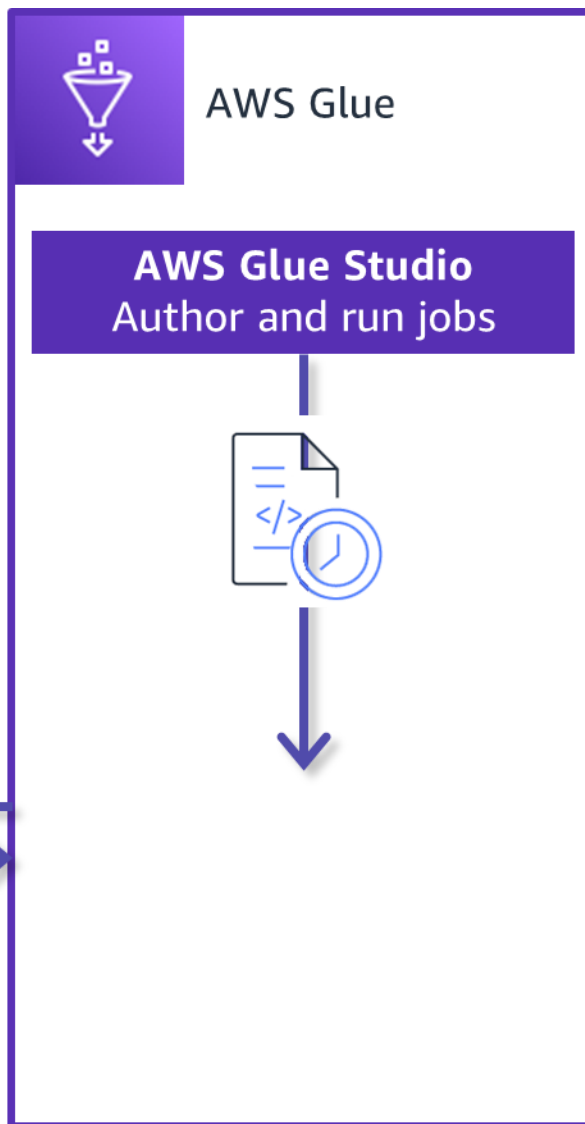


Schema identification and data cataloging

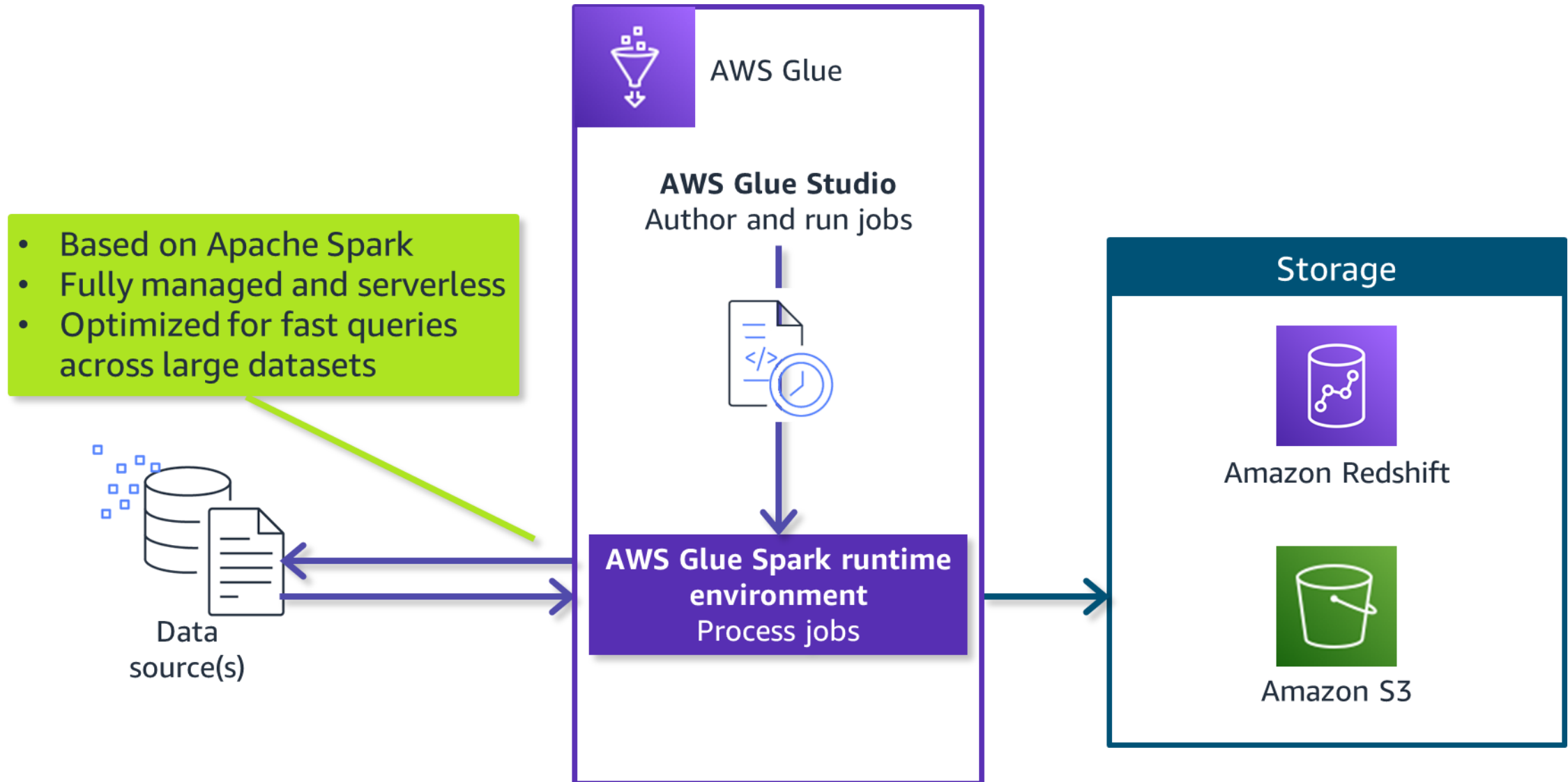


Job authoring

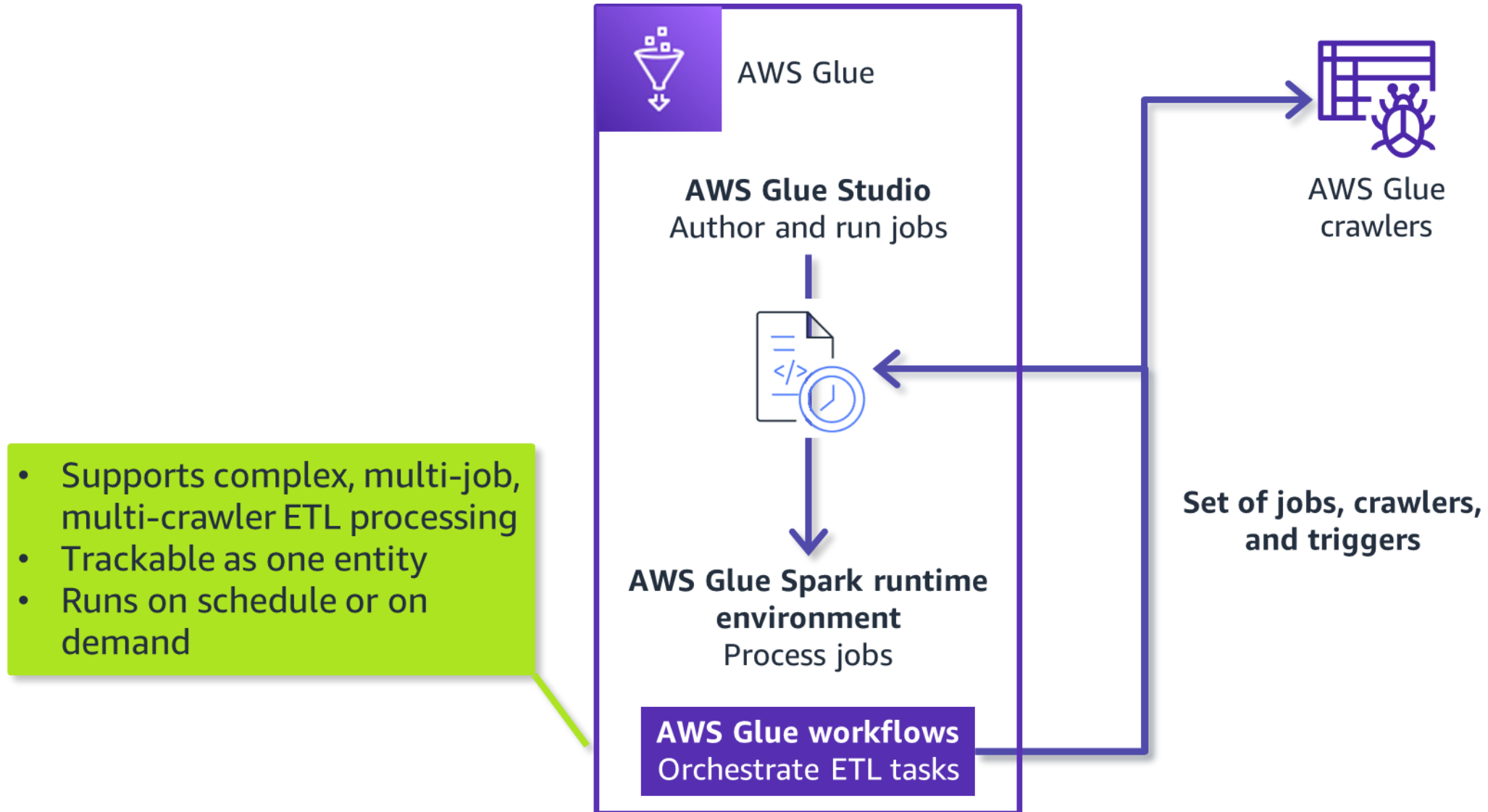
- Low-code job creation and management
- Graphical interface
- Transformations
- Monitoring



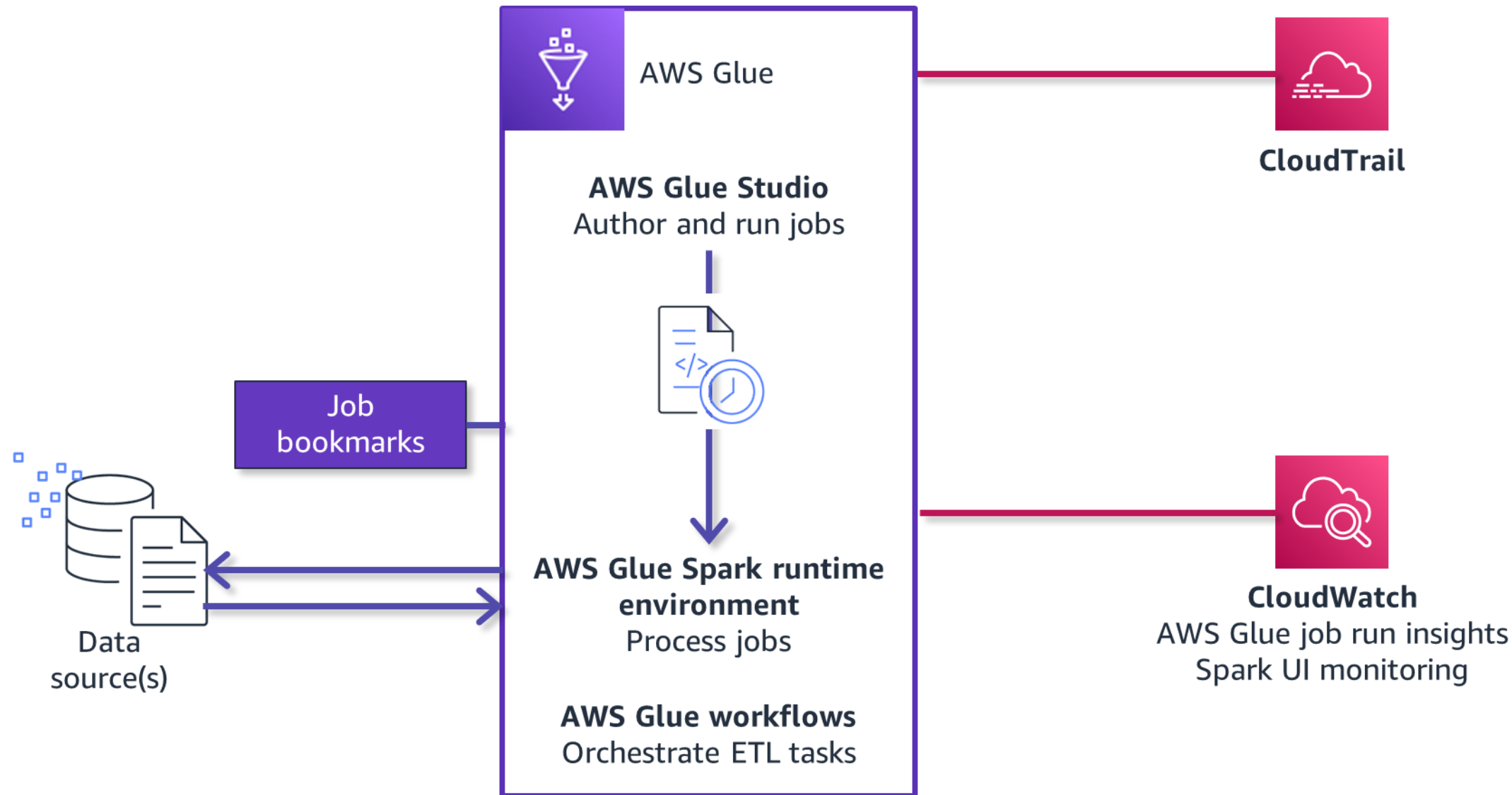
Serverless job processing



ETL orchestration



Monitoring and troubleshooting AWS Glue jobs



Key takeaways: AWS Glue for batch ingestion processing



- AWS Glue is a fully managed data integration service that simplifies ETL tasks.
- AWS Glue crawlers derive schemas from data stores and provide them to the centralized AWS Glue Data Catalog.
- AWS Glue Studio provides visual authoring and job management tools.
- The AWS Glue Spark runtime engine processes jobs in a serverless environment.
- AWS Glue workflows provide ETL orchestration.
- CloudWatch provides integrated monitoring and logging for AWS Glue, including job run insights.

Scaling considerations for batch processing

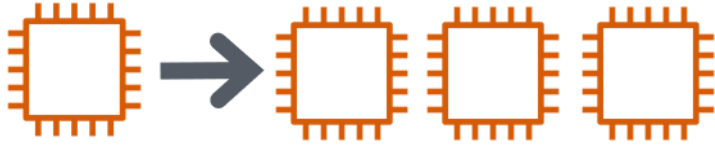
Ingesting by Batch or by Stream

Scaling and cost management

- Identify the key performance goals for your workload.
 - Complete within a specific time frame
 - Stay under a specific budget
 - Encounter fewer than X% of errors
- Use metrics to find issues and tune performance.
 - Steps that run for too long
 - Steps the use up too many resources
 - Failing steps



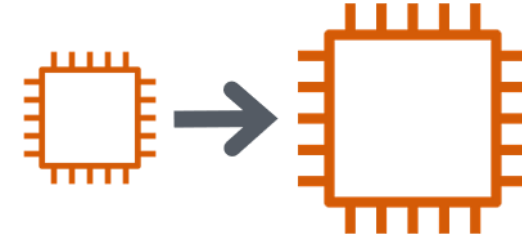
Using AWS Glue workers for horizontal and vertical scaling



Horizontal scaling

Increase the number of workers that are allocated to the job.

- **Use case:** Working with large, splittable datasets
- **Example:** Processing a large .csv file



Vertical scaling

Choose a worker type with larger CPU, memory, and disk space.

- **Use case:** Working with memory-intensive or disk-intensive applications
- **Example:** Machine learning (ML) transformations

Trade-offs of file size and compression codec choices



Many small files

- +** Work is distributed
- Higher overhead for the component that manages file to task distribution



Large, splittable file

- +** Lower overhead for file management
- +** Tasks can be distributed



Large, unsplitable file

- +** Only one file to manage
- Processing cannot be distributed

Key takeaways: Scaling considerations for batch processing



- Performance goals should focus on what factors are most important for your batch processing.
- Scale AWS Glue jobs horizontally by adding more workers.
- Scale AWS Glue jobs vertically by choosing a larger type of worker in the job configuration.
- Large, splittable files let the AWS Glue Spark runtime engine run many jobs in parallel with less overhead than processing many smaller files.

Lab: Performing ETL on a Dataset by Using AWS Glue

Lab introduction: Performing ETL on a Dataset by Using AWS Glue



- In this lab, you will work with AWS Glue to perform ETL on a dataset.
- You can direct AWS Glue to a data source, and it can infer a schema based on the data types that it discovers. Then, AWS Glue builds a Data Catalog, which contains metadata about the various data sources.
- Open your lab environment to start the lab and find additional details about the tasks that you will perform during this lab.

Debrief: Performing ETL on a Dataset by Using AWS Glue

- When you built the AWS Glue crawler in the first task, you had to specify a role for the crawler to use. Why does a crawler need an AWS Identity and Access Management (IAM) role to function inside AWS?
- What is an advantage of building an AWS Glue crawler and running it on demand or on a regular schedule?
- Which blockers exist for using this type of workflow with a team?

Services for stream processing

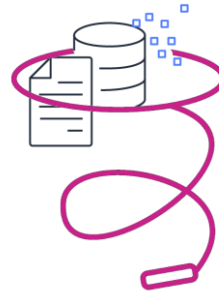
Ingesting by Batch or by Stream

Tasks to build a real-time stream processing pipeline

Extract

Transform/Load

Load/Transform



Streaming
events

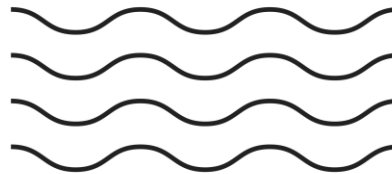
Put records on the
stream
(Producers)

Provide secure,
durable storage

Get records off the
stream
(Consumers)

Transform records
(Consumers)

Analyze or store
processed data



Data moves through the pipeline continuously

Key characteristics for stream ingestion and processing

Throughput

Plan for a resilient, scalable stream that can adapt to changing velocity and volume.

Loose coupling

Build independent ingestion, processing, and consumer components.

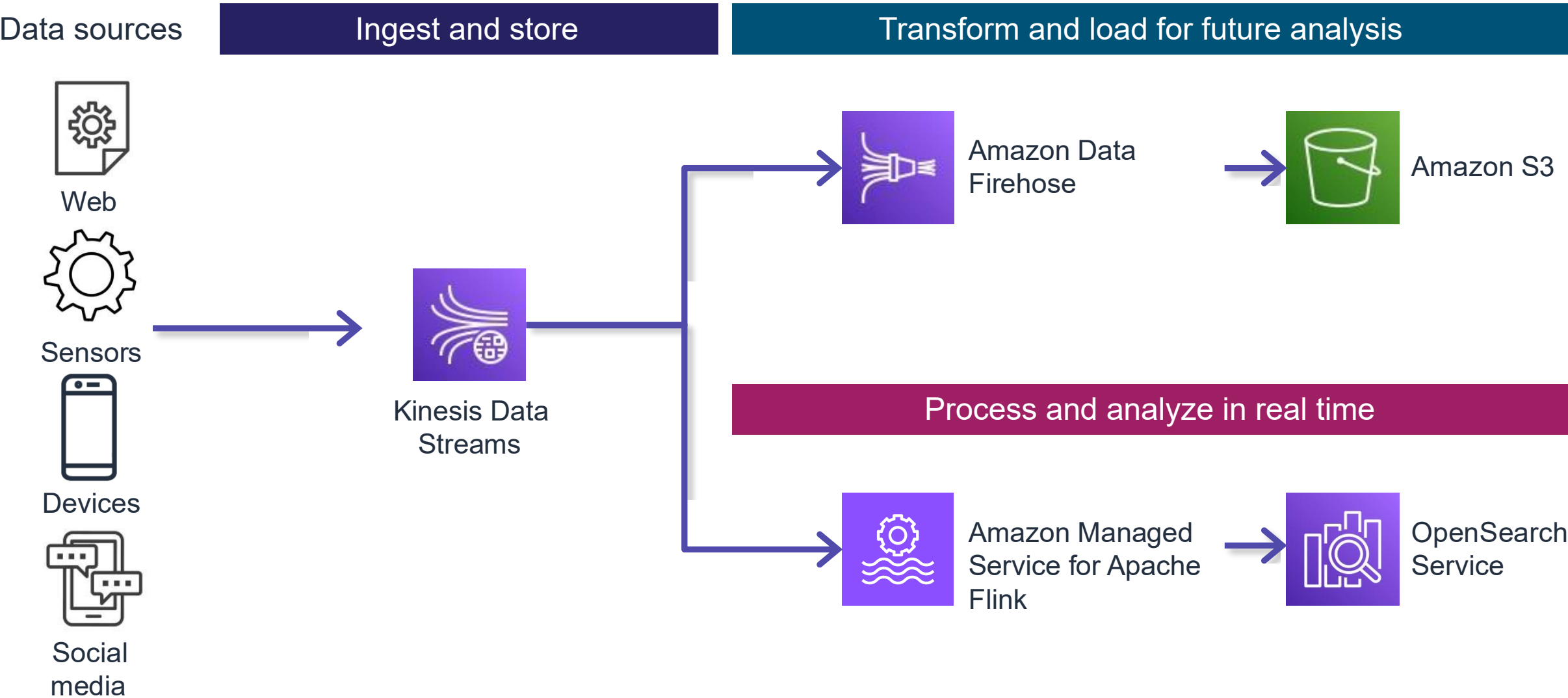
Parallel consumers

Allow multiple consumers on a stream to process records in parallel and independently.

Checkpointing and replay

Maintain record order and allow replay. Support the ability to mark the farthest record processed on failure.

Purpose-built streaming services simplify ingestion

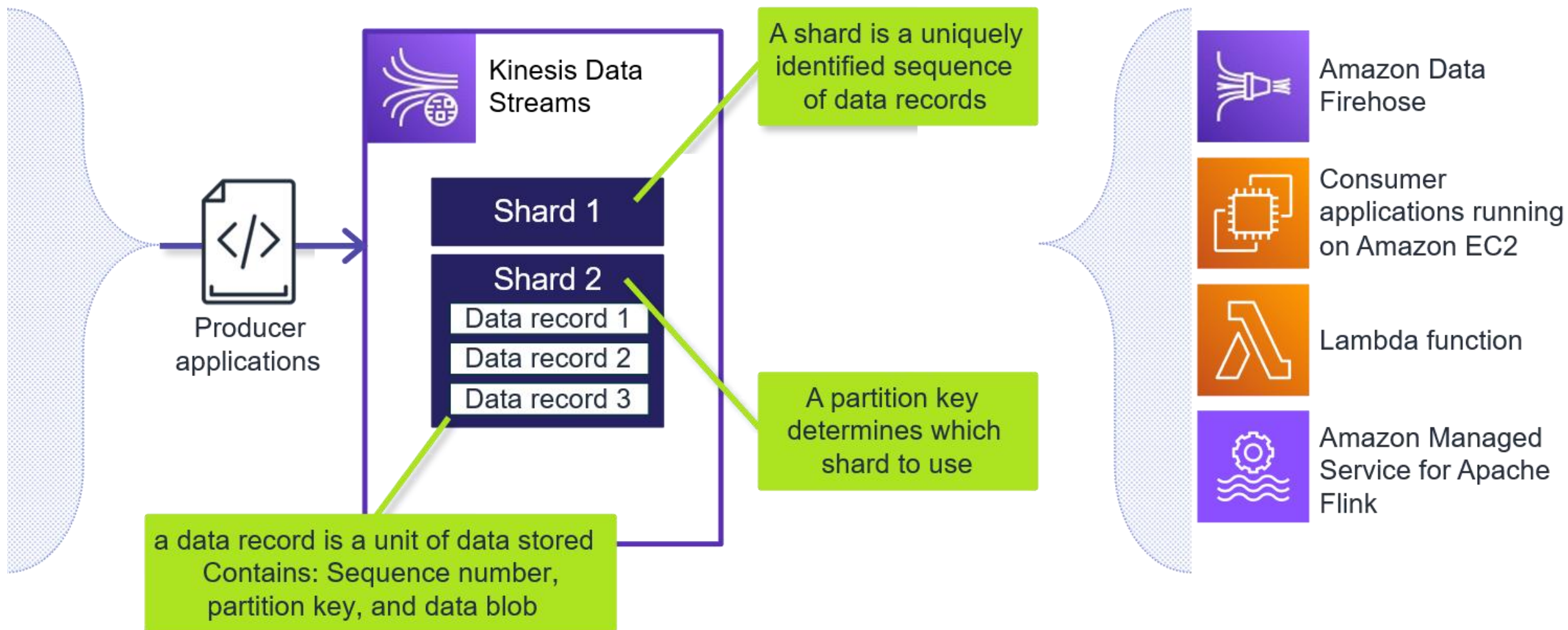


Kinesis Data Streams

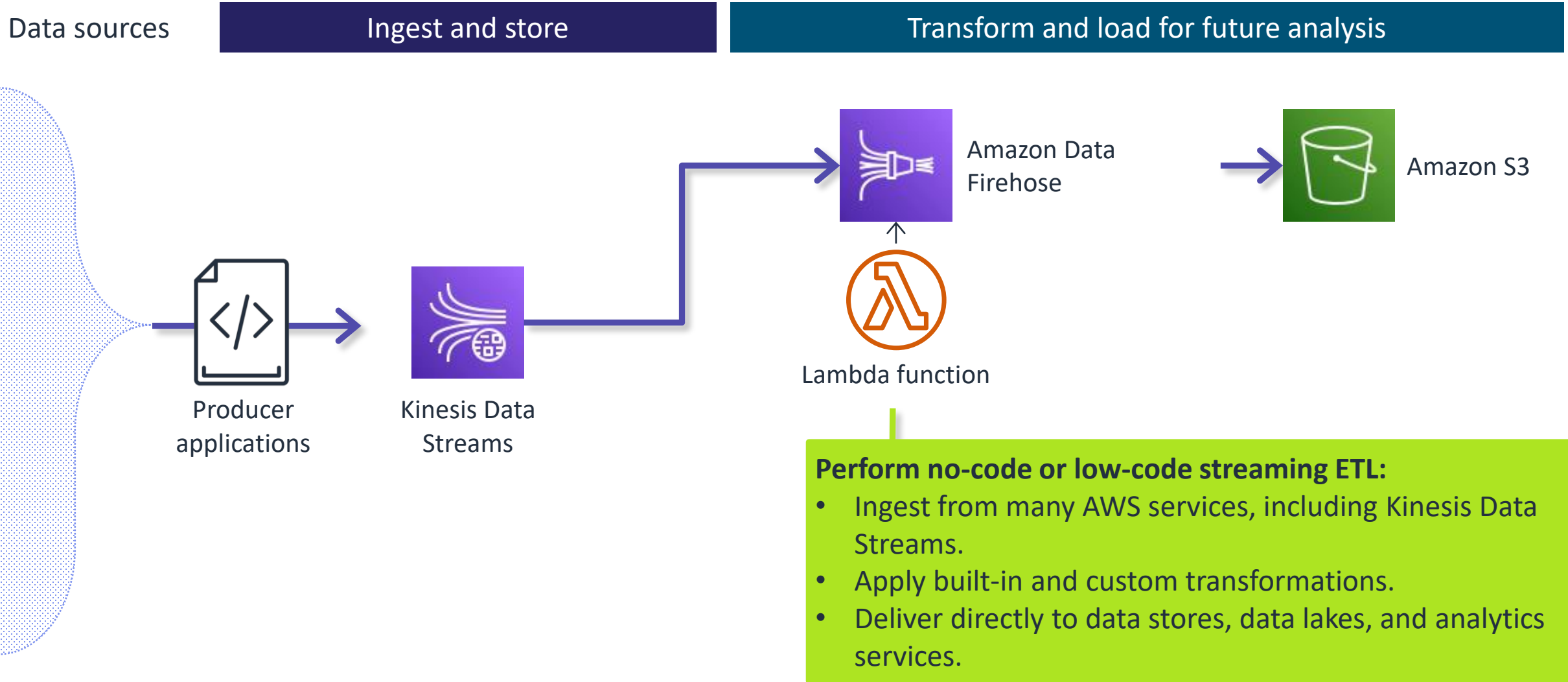
Producers

Ingest and store

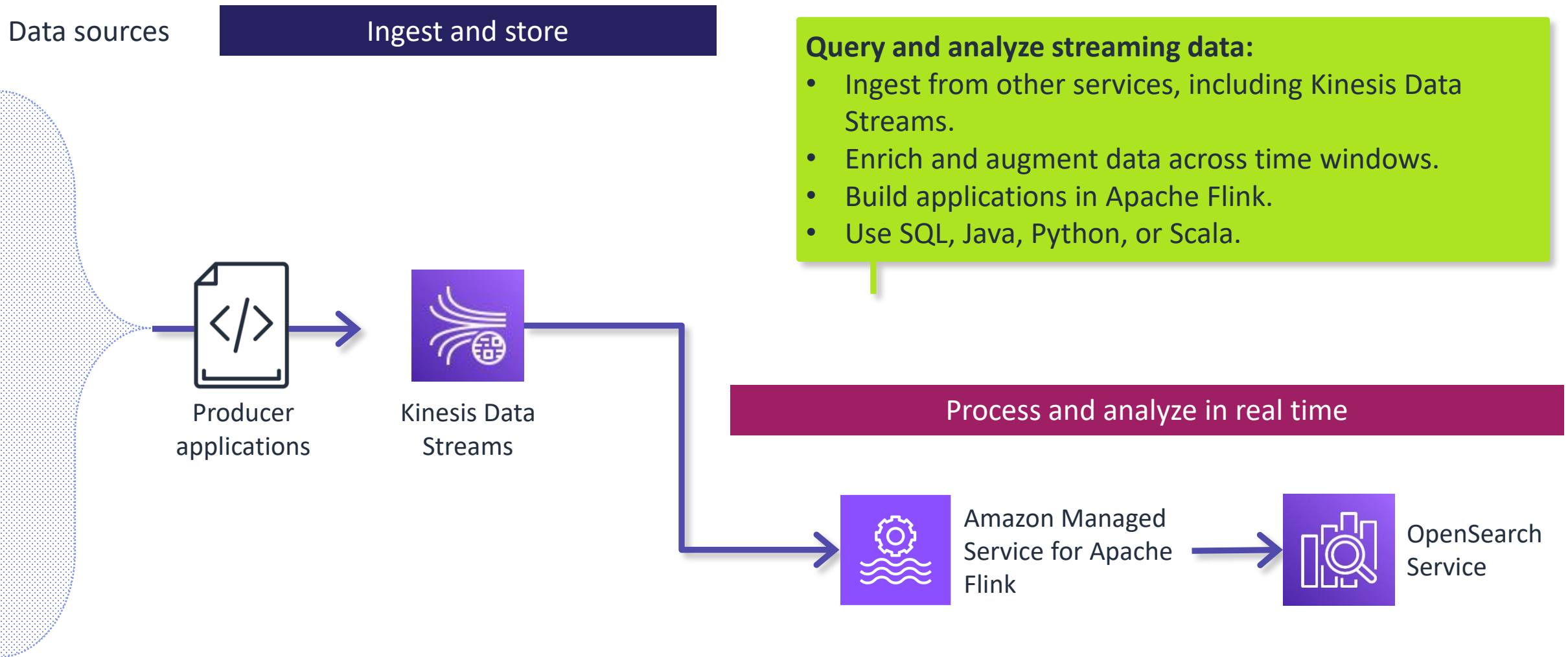
Consumers



Amazon Data Firehose



Amazon Managed Service for Apache Flink



Key takeaways: Services for stream processing



- The stream is a buffer between the producers and the consumers of the stream.
- The KPL simplifies the work of writing producers for Kinesis Data Streams.
- Data is written to shards on the stream as a sequence of data records.
- Data records include a sequence number, partition key, and data blob.
- Amazon Data Firehose can deliver streaming data directly to storage, including Amazon S3 and Amazon Redshift.
- Amazon Managed Service for Apache Flink is purpose built to perform real-time analytics on data as it passes through the stream.

Scaling considerations for stream processing

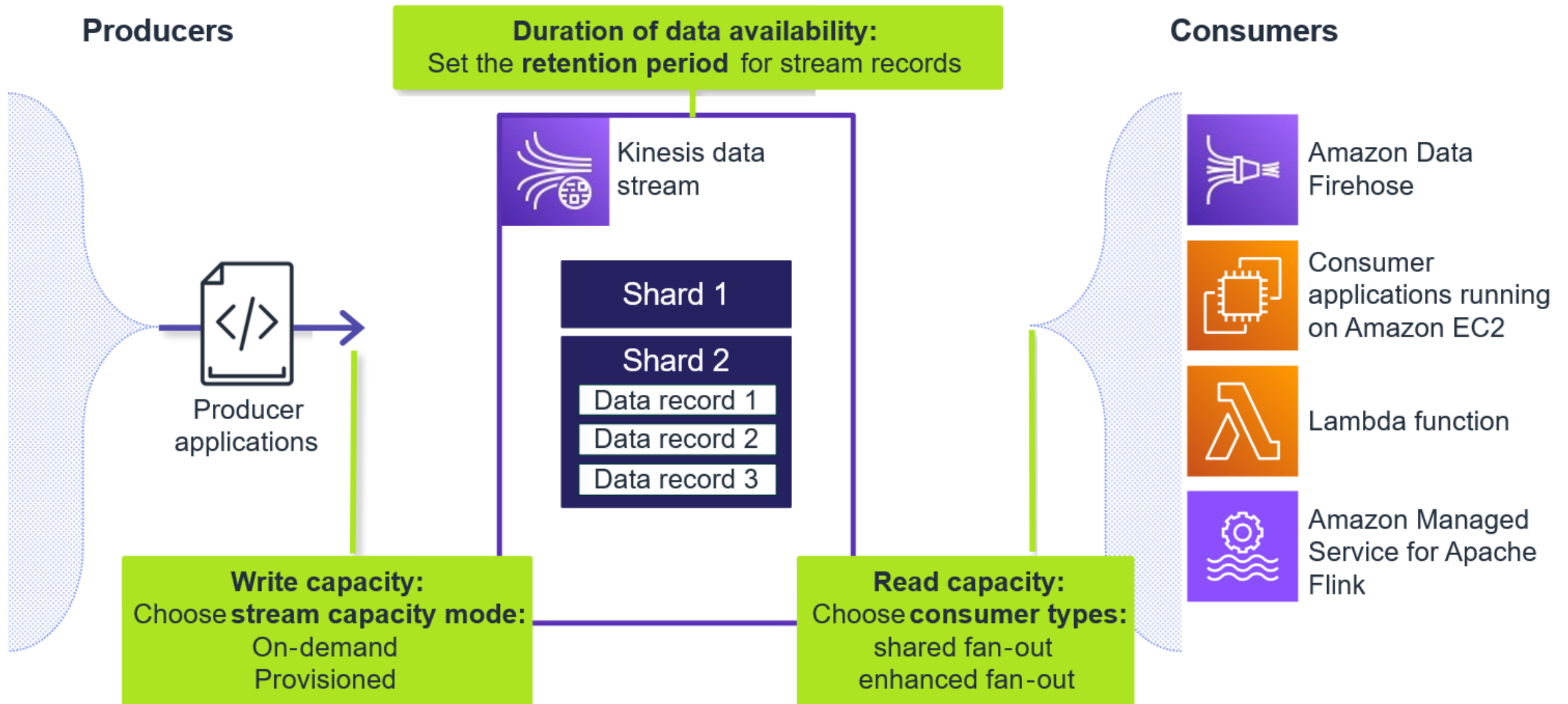
Ingesting by Batch or by Stream

Scaling and cost management with streams

- Configure the stream to meet your goals.
 - Set how long records will be available on the stream.
 - Scale write capacity for producers.
 - Scale read capacity for consumers.
- Use metrics to find issues and tune performance.
 - Check for throttling of producers or consumers.
 - Ensure records aren't expiring before they are processed.

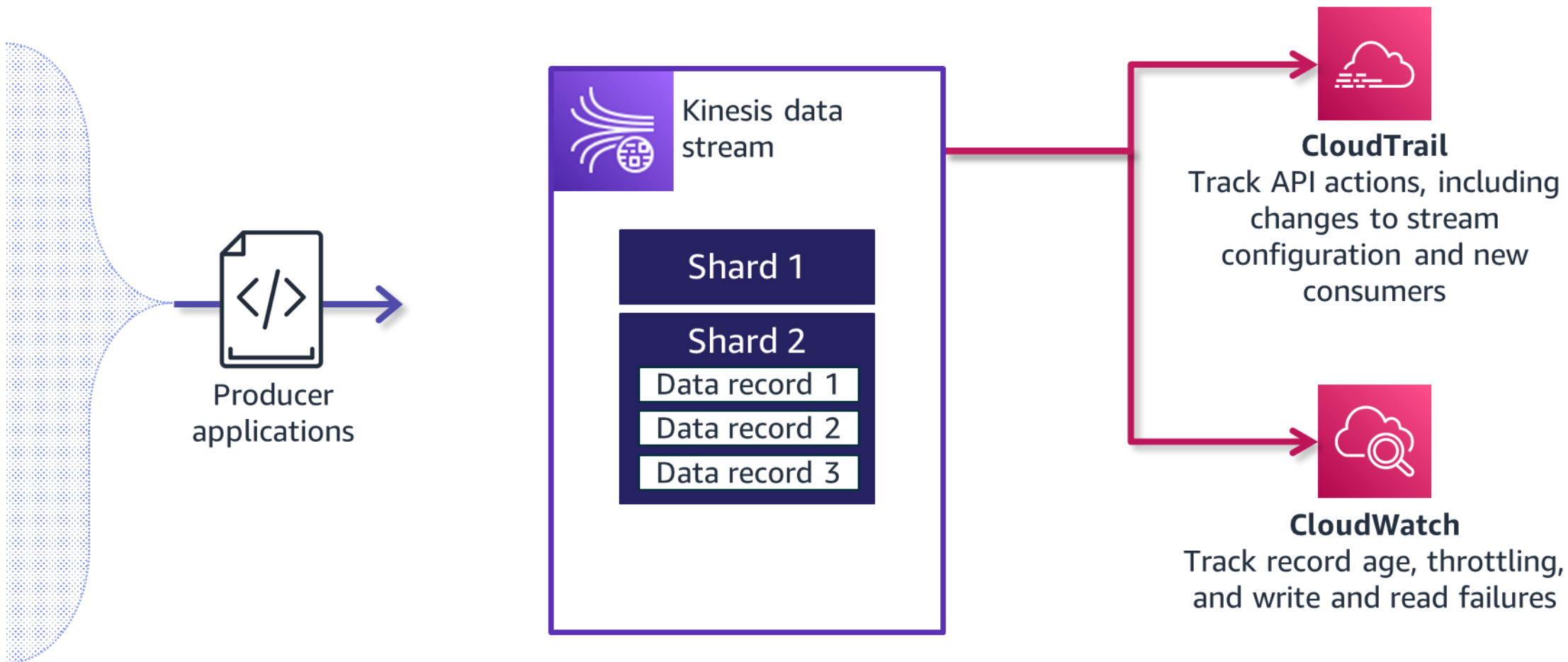


Three scaling configurations for Kinesis Data Streams



Monitoring a Kinesis data stream

Producers



Key takeaways: Scaling considerations for stream processing

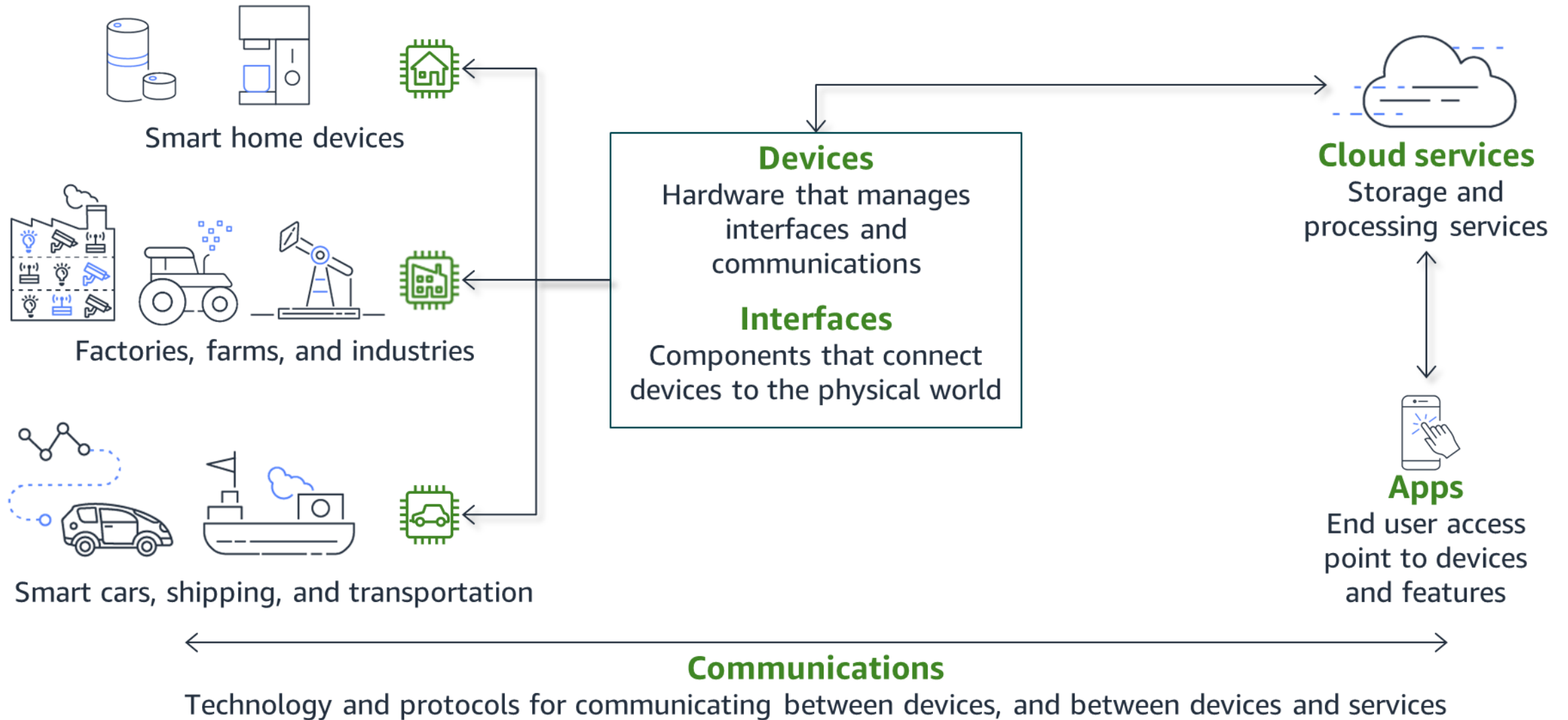


- Kinesis Data Streams provides scaling options to manage the throughput of data on the stream.
- You can scale how much data can be written to the stream, how long the data is stored on the stream, and how much throughput each consumer gets.
- CloudWatch provides metrics that help you monitor how your stream handles the data that is being written to and read from it.

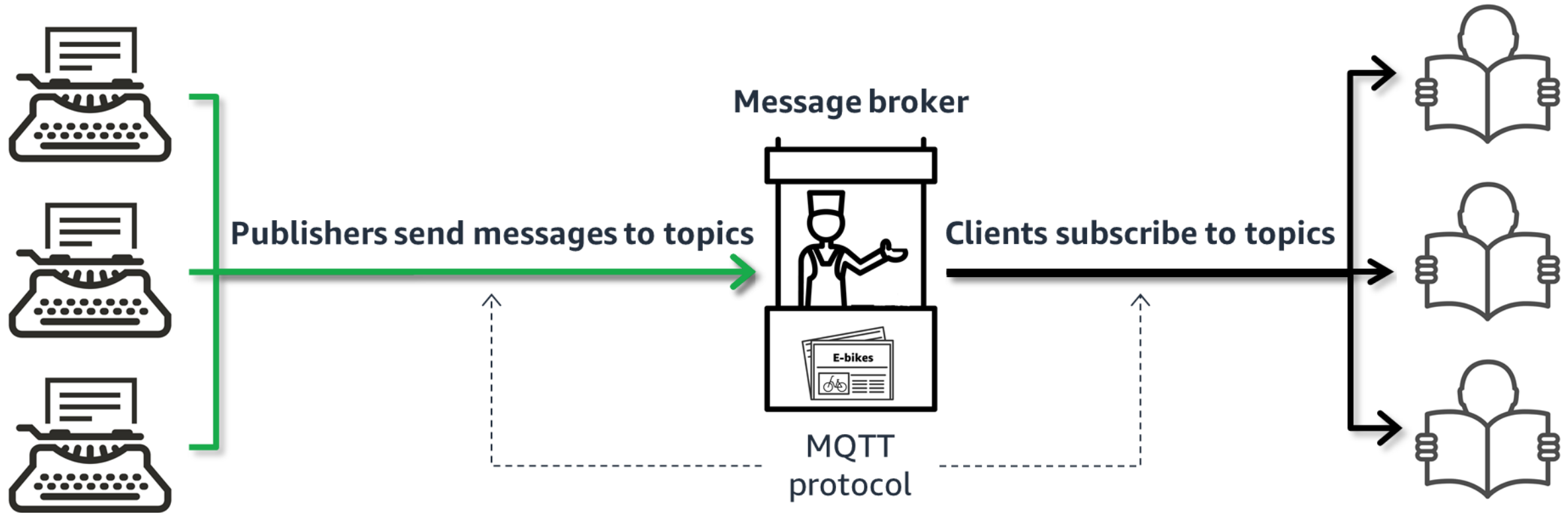
Ingesting IoT data by stream

Ingesting by Batch or by Stream

AWS IoT universe: Key components



MQTT and pub/sub for IoT device communications

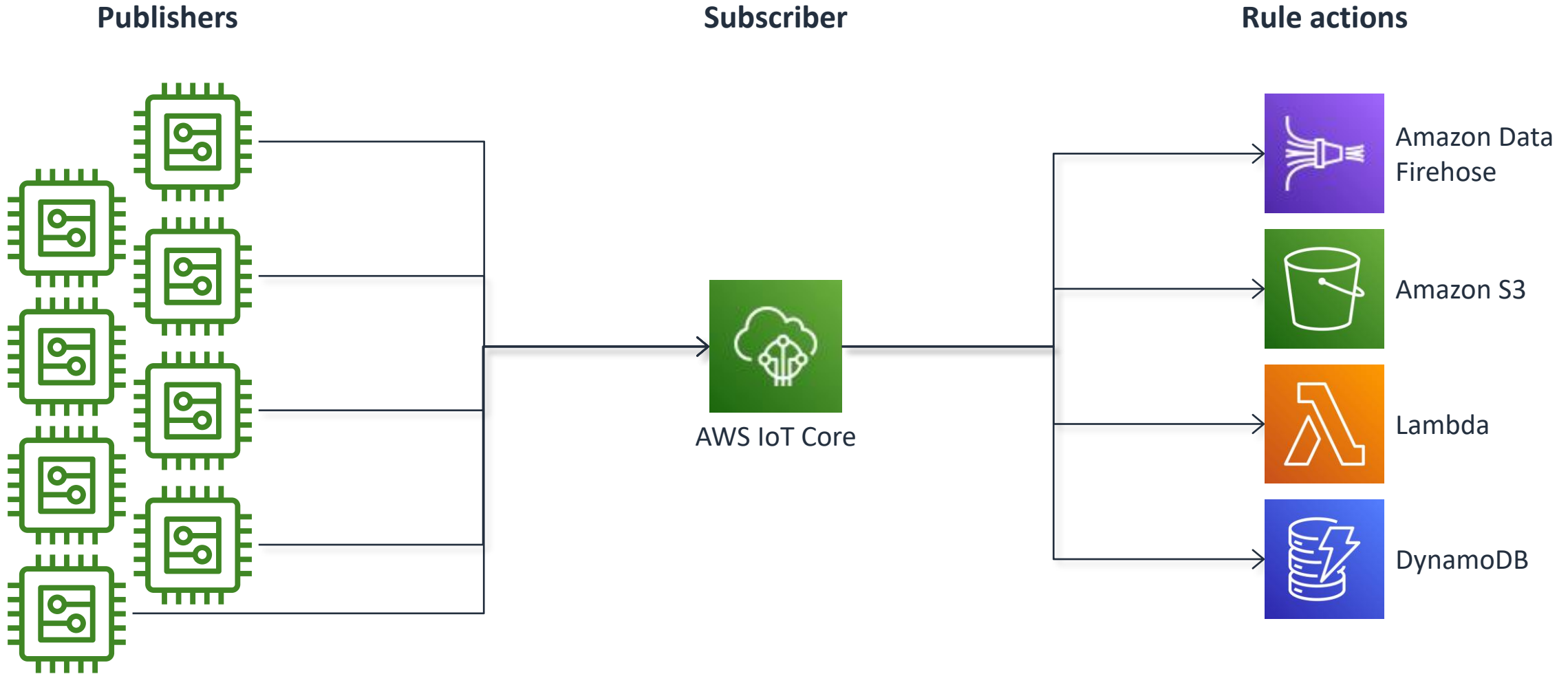


AWS IoT Core for data analytics and ML

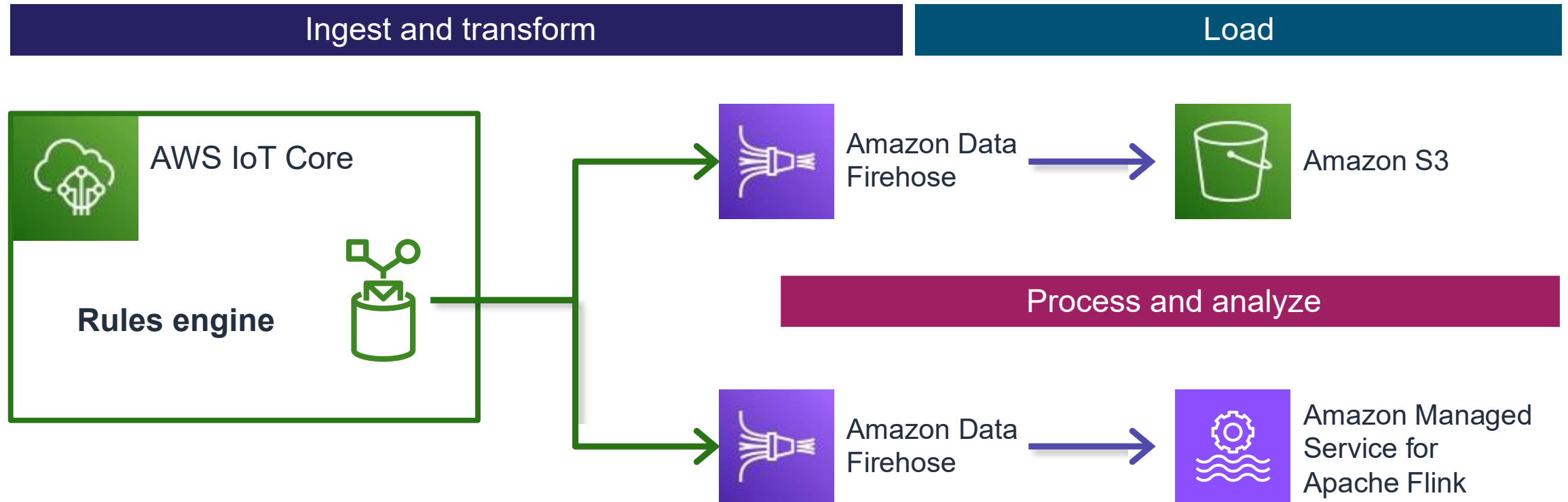
- Provides the ability to securely connect, process, and act on IoT device data
- Includes features to filter and transform data
- Can route data to other AWS services, including streaming and storage services



AWS IoT Core components for ingestion



The rules engine routes and transforms data



Key takeaways: Ingesting IoT data by stream



- With AWS IoT services, you can use MQTT and a pub/sub model to communicate with IoT devices.
- You can use AWS IoT Core to securely connect, process, and act upon device data.
- The AWS IoT Core rules engine transforms and routes incoming messages to AWS services.

Module wrap-up

Ingesting by Batch or by Stream

Module summary

This module prepared you to do the following:

- List key tasks that the data engineer needs to perform when building an ingestion layer.
- Describe how purpose-built AWS services support ingestion tasks.
- Illustrate how the features of AWS Glue work together to support automating batch ingestion.
- Describe AWS streaming services and features that simplify streaming ingestion.
- Identify configuration options in AWS Glue and Kinesis Data Streams that help you scale your ingestion processing to meet your needs.
- Describe distinct characteristics of ingesting IoT data by using AWS IoT Core.

Module knowledge check



- The knowledge check is delivered online within your course.
- The knowledge check includes 10 questions based on material presented on the slides and in the slide notes.
- You can retake the knowledge check as many times as you like.

Sample exam question

A data engineer is creating a stream processing pipeline that needs to reformat incoming data from .csv to .json before delivering it to an S3 bucket. They are looking for the option that requires the least amount of coding.

Which service should the data engineer use for this use case?

Identify the key words and phrases before continuing.

The following are the key words and phrases:

- **Stream** processing
- **Reformat** incoming data
- Delivering it to an **S3 bucket**
- **Least amount** of coding

Sample exam question: Response choices

A data engineer is creating a **stream processing** pipeline that needs to **reformat** incoming data from .csv to .json before delivering it to an **S3 bucket**. They are looking for the option that requires the **least amount of coding**.

Which service should the data engineer use for this use case?

Choice	Response
A	Use Amazon Kinesis Data Streams.
B	Use an Amazon Managed Service for Apache Flink application.
C	Use Amazon Data Firehose.
D	Use the Amazon Kinesis Client Library (KCL).

Sample exam question: Answer

The correct answer is C.

Choice	Response
--------	----------

C	Use Amazon Data Firehose.
---	---------------------------

Thank you



Corrections, feedback, or other questions?

Contact us at <https://support.aws.amazon.com/#/contacts/aws-academy>.

All trademarks are the property of their owners.